

GitHub Copilot 推進1年半の振り返りと AI駆動開発の実現へ向けて

Octonihon #6

NTTドコモソリューションズ 技術革新本部 システム技術部 DevTechセンタ

Kota Konishi | Jun 3, 2026

記載されている会社名、製品名、サービス名は、各社の商標または登録商標です。
記載の情報はNTTドコモソリューションズの公式見解ではなく個人の私見を多く含みます。

Key Takeaways

- 01 「はい、どうぞ」だけでは
「はい、よろこんで」にはならない
- 02 ガードレールで「審査不要」を実現
- 03 AI を使うこと自体がゴールではない
真のKGI は「速く・高品質・楽に」
- 04 ”動かない 100 点” より
”今この瞬間に動く 60 点”
- 05 AI 駆動開発は総合格闘技
- 06 意思決定がどこで行われるか
– それがすべてを決める

自己紹介



小西 航太

Kota Konishi

NTTドコモソリューションズ
技術革新本部
Systems Tech. Dept.
DevTechセンタ

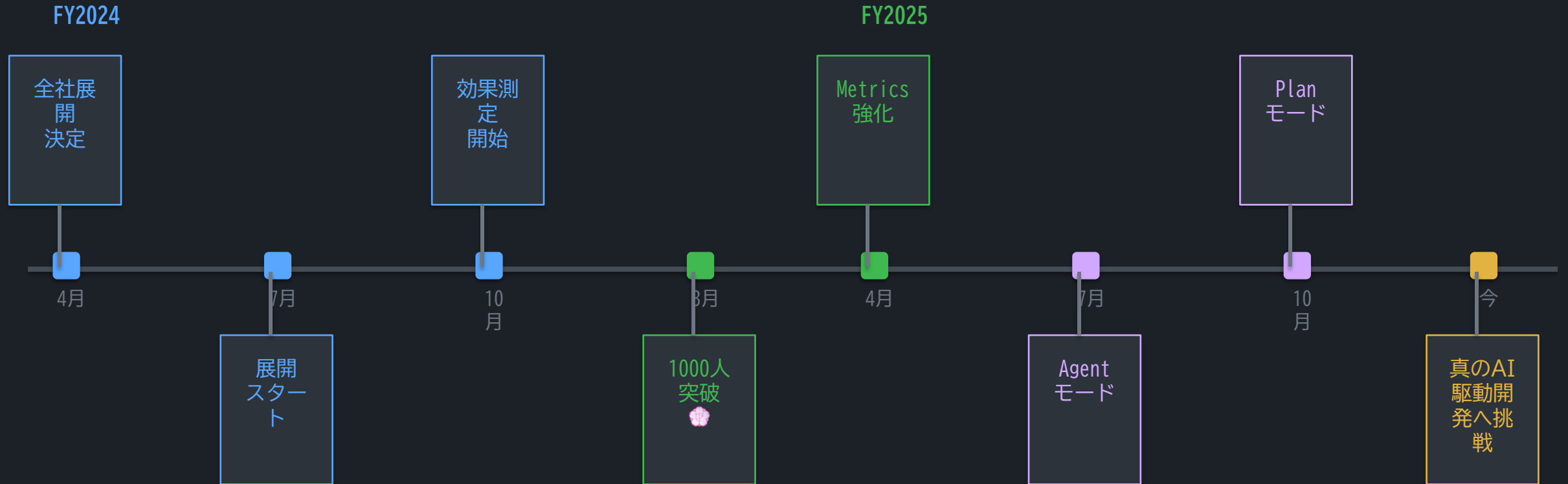
担当業務

- ▶ GitHub Copilot の全社展開・推進
- ▶ 開発ツール・技術の支援・普及
- ▶ 技術調査・評価
- ▶ 技術ガバナンス整備・推進

所属組織の役割

- ▶ システム開発技術・技法の研究と展開
- ▶ 開発ツール提供 SaaS PF の社内向け提供
- ▶ AI 活用のガードレール&アクセル整備
- ▶ 全社技術ガバナンス標準化・PJ 展開

ここまでの歩み



導入前の課題

「はい、どうぞ」だけでは「はい、よろこんで」にはならない



セキュリティリスク

- ▶ クラウド慎重な社風
- ▶ AI による情報流出懸念
- ▶ 意図しない学習の事例など



審査・手続きコスト

- ▶ 必要な審査 2～3件
- ▶ チェックリスト 3～4個
- ▶ マニュアル整備が必要



稼働・時間コスト

- ▶ 使い始めるだけで大工事
- ▶ 各組織への説明・調整
- ▶ 著作権・品質方針の策定

ガードレールを整える

想定する使い方の範囲内なら、審査しなくても使って良い！

Before

- ▶ セキュリティ審査 必要
- ▶ 個別対応で稼働大
- ▶ 使えるまで数ヶ月

整備したこと

- ▶ 審査範囲の明確化
- ▶ チェックリスト作成
- ▶ ガイドライン・マニュアル整備
- ▶ VS Code 限定でスタート
- ▶ Organization で設定管理

After

- ▶ 審査不要で利用開始
- ▶ 全社に安全に展開
- ▶ のべ 1000 人超が利用

GitHub Copilot を選んだ理由

OpenAI GPT

学習利用なし・データ保持なし

Google Gemini

学習利用なし・データ保持なし

Anthropic Claude

学習利用なし・データ保持なし

GitHub Copilot

ゼロデータ保持

Zero Data Retention

各モデルとGitHubが契約
入力データを学習・保持しない

Trust Center

公式トラストセンターで
リスク評価が容易

GitHub Enterprise 構成

ドコモからライセンス調達 → 個別 Organization で自社ルールに合わせた設定を実現

Enterprise: NTT Docomo SCE

Organization: `nttcw-copilot-users`

自社ルールに合わせた設定・ポリシー管理

Team:
PJ-A



PJ コードで識別

Team:
PJ-B



PJ コードで識別

Team:
PJ-C



PJ コードで識別

Team:
...



PJ コードで識別

→ Team slug の数だけ Metrics API を叩いてプロジェクト別効果測定が可能に

効果測定 – Copilot Metrics API

「効果どうなってんのよ」に答えるために独自実装したデータ基盤



Copilot
Metrics API

GitHub が提供
28日分のデータ



定期取得
スケジューラー

API を定期実行
データを逐次取得



独自 DB
(長期蓄積)

28日制限を突破
長期トレンド分析可能



ダッシュボード
可視化

承認率・利用者数
PJ別・期間別表示

未来の話 – 次のフェーズへ

GitHub Copilot の普及期がひと段落して見えてきたこと

ドキュメント

- ▶ テスト仕様書の自動生成
- ▶ 指示書のテンプレート化
- ▶ Issue × ドキュメント連携

現在地

ソースコード補完
が当たり前になった

上流工程

- ▶ 要件・設計フェーズでの活用
- ▶ Issue ベースの開発促進
- ▶ Copilot 活躍領域の拡大

目指す姿

ソースコード × Issue × ドキュメントを隣人化し、GitHub Copilot が活躍できる領域を左右に広げる。
「書くだけ」から「考える・設計する・振り返る」フェーズへ

まとめ

GitHub Copilot 推進 1年半 - キーポイント



ガードレールと全社展開

- ▶ セキュリティ懸念を審查明確化で解消
- ▶ 審査不要・全社に安全に展開
- ▶ のべ 1,000 人超が利用
- ▶ Copilot 選定: Zero Data Retention
- ▶ 全モデルで学習・保持なし



効果測定と普及の課題

- ▶ Metrics API × 独自 DB で長期蓄積
- ▶ PJ 別承認率・利用者数を可視化
- ▶ 現場セミナー・草の根普及が鍵



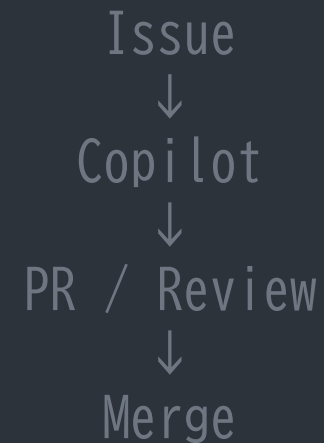
Agent × Plan と次フェーズ

- ▶ Agent モード: 自律的にコードを書く
- ▶ Plan モードで事前レビュー → 安全
- ▶ 次: コード × Issue × ドキュメントを隣人化
- ▶ 上流(要件・設計)へ活躍領域を拡大
- ▶ 「書くだけ」→ 「考える・設計する」へ

Section 2

AI 駆動開発の姿

目指している
GitHub × AI 開発の全体像



なぜ AI 駆動開発をするのか

AI を使っていること自体がゴールではない

❌ 「AI 活用率が高い」「Copilot を入れた」はゴールではなく、あくまで手段

❌ ありがちな誤ったゴール設定

- ▶ 「Copilot を全員に配った」
- ▶ 「AI Agent を動かしている」
- ▶ 「サジェスト承認率が XX%」
- ▶ 「AI 活用度スコアが高い」

これらは KPI(測定指標)であって
KGI(最終ゴール)ではない



✅ 真の KGI

- 🚀 今よりも開発が速くなること
- ★ 今よりも品質が高くなること
- 😊 今よりもチームが楽になること

これが達成できているか？

→ AI はそのための道具の一つ

AI 駆動開発は総合格闘技

技術だけでも、プロセスだけでも勝てない – 全部の武器が揃って初めて機能する



AI ツール活用

- ▶ Copilot / Claude など使い分け
- ▶ プロンプトエンジニアリング
- ▶ MCP でコンテキスト拡張



GitHub ワークフロー

- ▶ Issue 駆動・ADR・PR 文化
- ▶ ブランチ戦略・保護ルール
- ▶ AI と人間でブランチを分離



自動化 & 品質保証

- ▶ CI/CD パイプライン整備
- ▶ Coding Agent の運用
- ▶ テスト・ドキュメント自動化



組織論 & 推進戦略

- ▶ 草の根 vs トップダウン設計
- ▶ プロダクト開発の組織体制
- ▶ 推進者の役割・巻き込み方



社内政治 & 調整

- ▶ 審査・承認プロセスの突破
- ▶ 関係部門の懸念を先に潰す
- ▶ 会社の技術戦略との関連付け



セキュリティ & ガバナンス

- ▶ データ保持・学習ポリシー確認
- ▶ 利用ガイドライン・ガードレール
- ▶ Organization 単位でポリシー管理

AI 駆動開発がうまくいく条件 – VS Code を参考にすると

意思決定がどこで行われるか – それがすべてを決める

✓ microsoft/vscode

 **意思決定の場** GitHub Issues / PR / Discussion
→ すべてが GitHub 上で完結・公開

 **AI のアクセ
ス性** MCP 経由で Issue・PR・コード・
Discussion を一括参照可能

 **貢献フロー** Issue → PR → CI/CD → Review
→ 明確・標準化・自動化しやすい

 **意思決定構造** フラット。同じフローで誰でも参加
→ AI もそのまま参加できる

torvalds/linux(一事例として)

 **意思決定の場** メーリングリスト (LKML) とメール
→ 長年の実績ある分散型レビュー文化

 **AI のアクセ
ス性** 文脈がメールアーカイブに蓄積
→ AI より人間の深い専門知識が主役

 **貢献フロー** git format-patch → メール送信
→ 厳格な品質管理・専門家文化が前提

 **意思決定構造** 階層型メンテナ制+Linus が最終判断
→ 超大規模・超高品質に最適化された構造

成功の鍵 – 「意思決定が迅速にできる範囲」でリポジトリ戦略をスタートさせる

まず自分たちが GitHub 上で速く決定できる小さな単位から始め、その範囲を少しずつ広げる。Linux カーネルは世界最高水準のOSSだが、目指すゴール（スピード・AI 活用）が異なる。自分たちのゴールに合った型から始めることが大切。

リーダーとしてのスタンス – 100 点と 60 点を使い分ける

AI 駆動開発を前に進めるために

⚡ ”動かない 100 点” より ”今この瞬間に動く 60 点” の方が価値があることが多い

💡 100 点の理想を語る

- ▶ 常に「AI 駆動開発の完成形」を語り続ける
- ▶ チームが目指す方向性のアンカーになる
- ▶ チームは 100 点を目指して奮闘する
- ▶ ただし、制約・審査・調整により時間がかかることもある
- ▶ それでも理想を下げない
 - 方向だけは常に正しく保つ

VS

⚡ 60 点を今この瞬間に届ける

- ▶ AI 駆動開発の環境もスピーディーに進化が必要
- ▶ リーダーが自ら「動く 60 点」を作って届ける
- ▶ 60 点でも動くことで、チームが前進できる
- ▶ 完璧でなくていい – まず動かすことでフィードバックと改善が生まれる
- ▶ リーダーが率先して「不完全でも出す」を体現

AI 駆動開発ブートストラップリポジトリ

開発環境の土台をここに集約



GitHub 環境整備

- ▶ リポジトリ作成・アクセス設定
- ▶ Branch Protection ルール設定
- ▶ CI/CD パイプライン構築
- ▶ Issue テンプレート(人間/AI 両対応)
- ▶ PR テンプレート整備



AI 活用ルール整備

- ▶ copilot-instructions.md
→ AI への振る舞い指示書
- ▶ プロンプトテンプレート集
- ▶ ADR で設計判断を記録・蓄積
- ▶ ブランチ命名規則(人間/AI 分離)



自動化エージェント

- ▶ Issue 自動トリアージ
- ▶ 日次ドキュメント更新 PR 作成
- ▶ CI 失敗自動調査(CI Doctor)
- ▶ Issue アサイン → Slack 通知
- ▶ デイリー進捗レポート生成

AI 駆動開発ワークフロー

人起点・AI 自律・継続改善の 3 軸を同時に回す

🌐 GitHub をすべての中心に – Issue・PR・コード・ドキュメントをここに集約



Step 1 Issue 起票 / Assign

- ▶ 人が Issue を起票
- ▶ 定型タスクは Copilot に Assign
- ▶ 設計が必要なら ADR を作成
- ▶ MCP 経由でコンテキスト参照



Step 2 PR 作成・レビュー

- ▶ Copilot が実装 → PR 自動作成
- ▶ 人間 + Copilot でコードレビュー
- ▶ CI/CD Gate で品質を自動保証
- ▶ 承認後マージ → リリース作成



日次 AI エージェント(並行)

- ▶ ドキュメント差分 → 更新 PR 作成
- ▶ Issue トリアージ(ラベル/優先度)
- ▶ 進捗レポート生成 → Chat 通知
- ▶ Dependabot / Security 修正 PR

目指す姿 – GitHub × AI で開発プロセスを変える

コード補完はスタート地点。人が「何を作るか」に集中できる世界へ

今の姿(コード補完期)

- ▶ AI はあくまでアシスタント
- ▶ 人がコードをほぼ全部書く
- ▶ ドキュメントは別管理で陳腐化
- ▶ レビューの工数が大きい
- ▶ Copilot の活躍 = IDE の中だけ
- ▶ 上流(要件・設計)は人頼み



目指す姿(AI 駆動開発)

- ▶ Issue 起票 → Copilot が自律実装 → PR
- ▶ 人はレビュー・設計・意思決定に集中
- ▶ ドキュメントは AI が自動更新

- ▶ 「書くだけ」→「考える・設計する」へ
- ▶ ブートストラップ で実践中

→ KGI 達成:  速く /  高品質 /  チームが楽に

Key Takeaways

- 01 「はい、どうぞ」だけでは
「はい、よろこんで」にはならない
- 02 ガードレールで「審査不要」を実現
- 03 AI を使うこと自体がゴールではない
真のKGI は「速く・高品質・楽に」
- 04 ”動かない 100 点” より
”今この瞬間に動く 60 点”
- 05 AI 駆動開発は総合格闘技
- 06 意思決定がどこで行われるか
– それがすべてを決める